Esercizi — shell e file system

Lezione coperta: `L02` — informazione, calcolatore, SO, file system, shell

Prima di iniziare: serve la macchina virtuale del corso (`guida_VM.md`) e la lettura di `L02_shell.md`. Nessun esercizio usa comandi che non siano spiegati là.

Come si lavora. Apri il terminale e parti dalla tua *home*: il primo comando di ogni sessione è `cd`, che ti riporta lì da qualunque punto. Gli esercizi si **concatenano**: il 4 lavora su quello che ha costruito il 3, quindi vanno fatti in ordine. Se ti perdi, `pwd` dice dove sei e `ls` cosa hai intorno.

Le soluzioni sono qui sotto ogni esercizio, ma aprile **dopo** aver provato: il comando lo imparano le dita, non gli occhi. L'unica eccezione è l'ultimo, la Caccia al Tesoro: quello si risolve senza rete, ed è il modo di verificare se la lezione è entrata.

Esercizio 1 — La directory di lavoro (difficoltà: •)

Crea nella tua home una directory `E01`. Dentro `E01` crea due directory, `d01` e `d02`. Dentro `d02` crea `d02Sub`.

Poi verifica di aver costruito l'albero giusto: entra in `E01` e chiedi alla shell dove sei e cosa c'è.

> **Suggerimento:** `mkdir -p` crea anche le directory intermedie che mancano, e ti risparmia tre comandi.

Soluzione

Mostra la soluzione

```
cd                          # dalla home, sempre
mkdir E01
cd E01
mkdir d01
mkdir -p d02/d02Sub         # crea d02 e d02Sub in un colpo
pwd                         # /home/tuo.nome/E01
ls                          # d01  d02
```

Esercizio 2 — Un file vuoto, e uno con qualcosa dentro (difficoltà: •)

Dentro `d01` crea un file vuoto chiamato `file01`. Guarda con `ls -l` quanto occupa.

Poi scrivici tre righe — una alla volta — così che il contenuto diventi:

```
Benvenuti
al corso
di Informatica
```

Controlla di nuovo la dimensione: non è più zero.

> **Suggerimento:** `echo` scrive una riga, `>` la manda in un file nuovo (o sovrascrive quello che c'era), `>>` la aggiunge in fondo. Sbagliare `>` con `>>` qui costa poco: è esattamente il momento in cui conviene sbagliare.

Soluzione

Mostra la soluzione

```
touch d01/file01
ls -l d01/                          # dimensione 0

echo "Benvenuti"      >  d01/file01   # crea e scrive la prima riga
echo "al corso"       >> d01/file01   # aggiunge
echo "di Informatica" >> d01/file01   # aggiunge
ls -l d01/                            # ora la dimensione non e' piu' 0
```

Se al terzo comando avessi usato `>` invece di `>>`, il file conterrebbe solo `di Informatica`: `>` non aggiunge, sostituisce.

Esercizio 3 — Guardare dentro un file (difficoltà: •)

Stampa a video il contenuto di `d01/file01`. Poi stampa **solo la prima riga**, e poi **solo l'ultima**. Infine conta quante righe ha.

> **Suggerimento:** quattro comandi diversi, uno per domanda.

Soluzione

Mostra la soluzione

```
cat d01/file01              # tutto il contenuto
head -n 1 d01/file01        # Benvenuti
tail -n 1 d01/file01        # di Informatica
wc -l d01/file01            # 3 d01/file01
```

Esercizio 4 — Copiare e spostare (difficoltà: •)

1. Copia `d01/file01` in `d02`, mantenendo lo stesso nome.
2. Copia `d01/file01` in `d02/d02Sub`, ma chiamandolo `file02`.
3. Sposta `d02/file01` dentro `d02/d02Sub`.
4. Verifica: in `d01` il file ci deve essere ancora, in `d02` no, e in `d02/d02Sub` ce ne devono essere due.

> **Suggerimento:** `cp` lascia l'originale dov'è, `mv` no. Alla fine `ls -R E01` ti mostra tutto l'albero in un colpo.

Soluzione

Mostra la soluzione

```
cp d01/file01 d02/                    # stesso nome, altra directory
cp d01/file01 d02/d02Sub/file02       # nome nuovo
mv d02/file01 d02/d02Sub/             # sposta: in d02 non c'e' piu'

ls d01                                # file01
ls d02                                # d02Sub
ls d02/d02Sub                         # file01  file02
```

Esercizio 5 — Rinominare (difficoltà: •)

Rinomina `d02/d02Sub/file02` in `copia.txt`, senza spostarlo.

> **Suggerimento:** non serve un comando nuovo. Spostare e rinominare sono la stessa operazione.

Soluzione

Mostra la soluzione

```
mv d02/d02Sub/file02 d02/d02Sub/copia.txt
ls d02/d02Sub                         # copia.txt  file01
```

Esercizio 6 — Percorsi relativi e assoluti (difficoltà: ••)

Entra in `d02/d02Sub` e, **da lì**, stampa il contenuto di `d01/file01` in tre modi diversi: con un percorso assoluto, con un percorso che parte dalla tua home, e con un percorso relativo.

> **Suggerimento:** `~` è la home, `..` sale di un livello — e si può usare più di una volta.

Soluzione

Mostra la soluzione

```
cd d02/d02Sub
pwd                                   # /home/tuo.nome/E01/d02/d02Sub

cat /home/tuo.nome/E01/d01/file01     # assoluto (sostituisci tuo.nome)
cat ~/E01/d01/file01                  # dalla home
```

```
cat ../../d01/file01                # relativo: su di due, poi in d01

cd ~/E01                            # si torna alla base per gli esercizi seguenti
```

Tre indirizzi, un solo file. Il terzo funziona **solo** da `d02Sub`: è il senso di «relativo».

Esercizio 7 — Mettere due comandi in fila (difficoltà: ••)

Costruisci un file `d01/lungo.txt` che contenga i numeri da 1 a 20, una riga ciascuno. Poi, con un solo comando, stampa a video **la riga 11 e la riga 12**.

> **Suggerimento:** per il file, `seq`; per le due righe, `head` e `tail` collegati da una pipe. Le prime 12 righe, e di quelle le ultime 2.

Soluzione

Mostra la soluzione

```
seq 1 20 > d01/lungo.txt            # seq stampa una sequenza, > la salva

head -n 12 d01/lungo.txt | tail -n 2  # 11  12
```

La pipe passa al secondo comando l'output del primo: nessun file intermedio.

Esercizio 8 — Salvare il risultato di un comando (difficoltà: ••)

Conta le righe di `d01/lungo.txt` e salva il risultato in un file `numeroRighe01`, dentro `E01`. Poi conta le righe di `d02/d02Sub/file01` e **aggiungi** il risultato allo stesso file, senza perdere il primo.

Alla fine `numeroRighe01` deve contenere due righe.

> **Suggerimento:** è la differenza fra `>` e `>>` dell'esercizio 2, questa volta con l'output di un comando invece di `echo`.

Soluzione

Mostra la soluzione

```
wc -l d01/lungo.txt          >  numeroRighe01
wc -l d02/d02Sub/file01      >> numeroRighe01
cat numeroRighe01
#   20 d01/lungo.txt
#    3 d02/d02Sub/file01
```

Esercizio 9 — Cercare dentro i file (difficoltà: ••)

In `d01/file01` trova la riga che contiene la parola `corso`. Poi conta quante righe, in tutti i file sotto `E01`, contengono la parola `Benvenuti`.

> **Suggerimento:** `grep` cerca dentro i file; `-r` lo fa scendere nelle sottodirectory, `-c` conta invece di stampare.

Soluzione

Mostra la soluzione

```
grep "corso" d01/file01              # al corso
grep -r "Benvenuti" .                # elenca i file e le righe trovate
grep -rc "Benvenuti" .               # un conteggio per file
```

Attenzione alle maiuscole: `grep "benvenuti"` non trova niente. Con `-i` sì.

Esercizio 10 — Cercare i file (difficoltà: ••)

Trova, partendo da `E01`, tutti i file il cui nome inizia con `file`. Poi tutti quelli che finiscono con `.txt`.

> **Suggerimento:** qui non serve `grep` ma `find`, e l'opzione `-name` vuole il modello fra apici: `'file*'`.

Soluzione

Mostra la soluzione

```
find . -name 'file*'
# ./d01/file01
# ./d02/d02Sub/file01

find . -name '*.txt'
# ./d01/lungo.txt
# ./d02/d02Sub/copia.txt
```

`grep` cerca **dentro** i file, `find` cerca **i** file.

Esercizio 11 — Cancellare, con cautela (difficoltà: ••)

1. Crea una directory `d02/vuota` e cancellala con il comando che funziona solo sulle directory vuote.
2. Cancella il file `d02/d02Sub/copia.txt`.
3. Cancella la directory `d01` con tutto il suo contenuto.

Dopo il punto 3, `E01` deve contenere solo `d02` e `numeroRighe01`.

> **Attenzione:** non c'è cestino e non c'è conferma. Prima di ogni `rm -r`, `pwd` e `ls` — nell'ordine.

Soluzione

Mostra la soluzione

```
mkdir d02/vuota
rmdir d02/vuota                      # solo directory vuote

rm d02/d02Sub/copia.txt

ls d01                               # si guarda cosa si sta per perdere
rm -r d01                            # -r: la directory e tutto il contenuto

ls                                   # d02  numeroRighe01
```

`rmdir d01` avrebbe rifiutato, perché `d01` non era vuota. È una rete di sicurezza: quando `rmdir` protesta, fermati a guardare cosa c'è dentro.

Esercizio 12 — Comprimere e decomprimere (difficoltà: ••)

Comprimi la directory `E01` in un archivio `E01.zip`, mettendolo nella tua home. Elenca il contenuto dell'archivio **senza** decomprimerlo. Poi decomprimilo in una directory nuova, `~/verifica`, e controlla che l'albero sia quello giusto.

> **Suggerimento:** `zip -r` per le directory, `unzip -l` per guardare dentro, `unzip -d` per scegliere dove estrarre.

Soluzione

Mostra la soluzione

```
cd                                   # nella home
zip -r E01.zip E01                   # crea l'archivio

unzip -l E01.zip                     # elenca senza estrarre

mkdir verifica
unzip -q E01.zip -d verifica         # -q: senza l'elenco di tutto
ls -R verifica                       # l'albero e' quello di E01
```

`unzip -l` prima di estrarre è una buona abitudine: dice se l'archivio contiene una directory o cento file sciolti che ti riempiono quella corrente.

Esercizio 13 — I permessi (difficoltà: ••)

Crea nella home un file `saluta.sh` con questo contenuto:

```
echo "Ciao dalla shell"
```

Provalo con `./saluta.sh`: non funziona. Guarda i permessi con `ls -l`, rendilo eseguibile, riprova.

> **Suggerimento:** `chmod +x`. E `./` davanti al nome serve per dire «il file in questa directory», non un comando di sistema.

Soluzione

Mostra la soluzione

```
cd
echo 'echo "Ciao dalla shell"' > saluta.sh

./saluta.sh                  # bash: ./saluta.sh: Permission denied
ls -l saluta.sh              # -rw-r--r--  : nessun permesso di esecuzione

chmod +x saluta.sh
ls -l saluta.sh              # -rwxr-xr-x  : ora c'e' la x
./saluta.sh                  # Ciao dalla shell
```

Esercizio 14 — Lanciare Python (difficoltà: ••)

Crea una directory `~/script`, e dentro un file `primo.py` che stampi `Hello World`. Eseguilo **dalla home**, con un percorso relativo. Poi entra in `script` ed eseguilo di nuovo, questa volta senza percorso.

> **Suggerimento:** è lo stesso file, e i due comandi sono diversi. Il motivo è la directory corrente.

Soluzione

Mostra la soluzione

```
cd
mkdir script
echo 'print("Hello World")' > script/primo.py

python script/primo.py       # dalla home: serve il percorso
cd script
python primo.py              # da dentro: il nome basta
```

Se avessi scritto `python primo.py` dalla home, Python avrebbe risposto `can't open file 'primo.py'`. Non è un errore di Python: è un errore di percorso, e sarà il più frequente di tutto il corso.

Esercizio 15 — Fare ordine (difficoltà: •)

Cancella tutto quello che hai creato in questa esercitazione: `E01`, `E01.zip`, `verifica`, `saluta.sh`, `script`. Verifica che la home sia tornata come prima.

> **Suggerimento:** i file uno per uno, le directory con `-r`. Ed è il momento di **non** usare `rm -r *`.

Soluzione

Mostra la soluzione

```
cd
rm -r E01 verifica script
rm E01.zip saluta.sh
ls
```

`rm` accetta più argomenti in un comando solo. `rm -r *` avrebbe cancellato anche tutto il resto: è la ragione per cui gli argomenti si scrivono per nome.

Esercizio 16 — Caccia al tesoro (difficoltà: •••)

Nella cartella dei materiali trovi `CacciaAlTesoro.zip`. Decomprimilo e usa **soltanto** i comandi di questa lezione per trovare il tesoro.

Il punto di partenza è **il file non vuoto in `LaForesta`**: da lì ogni passo ti dice dove andare e cosa cercare. Il tesoro è dentro `IlCastello`, ma il castello si apre solo se trovi come abbassare il ponte levatoio.

Non c'è suggerimento e non c'è soluzione in questa scheda: è l'esercizio con cui si verifica se la lezione è entrata. Se ti blocchi, rileggi la tabella in `L02_riepilogo.md` e chiediti *quale domanda* stai facendo al file: cos'è, cosa contiene, quanto è lungo, dove sta.

> **Una cosa che non hai ancora visto** ti servirà alla fine: un file può essere *nascosto*. Su Unix i file il cui nome inizia con `.` non compaiono in `ls` — per vederli serve `ls -a`. Tienilo a mente quando sarai dentro il castello e ti sembrerà vuoto.